# CSS

# Day 8

## Display properties:

The display property in CSS specifies how an element should be rendered on the page

**display: block;**

- Element takes full width of its container (from left to right).
- Always starts on a new line.
- Examples: , <p>, <h1>…<h6>

**display: inline;**

- Element takes only as much width as its content.
- Does not start on a new line.
- Examples: , <a>, 

**display: inline-block;**

- Acts like inline (sits beside other elements) but also respects width, height, margin, padding (like block).
- Useful for navigation menus, buttons, cards.

**display: none;**

- The element is completely hidden from the page (it won't occupy any space).

- Different from visibility: hidden; (which hides content but keeps its space).

**display: flex;**

- Turns the element into a flex container.
- Child elements become flex items that can be arranged horizontally or vertically.
- Makes layout easier (navigation bars, grids, alignment).

**Display:grid;**

- Turns the container into a **grid layout system**.
- Children are placed in rows and columns.

**Examples:**

```html
<body>
    <p>Paragraph tag</p>
    <span>span tag</span>
    <span>span tag</span>
</body>
```

```css
p{
    width: 100px;
    height: 100px;
    border: 1px solid cadetblue;
}
span{
    width: 100px;
    height: 100px;
    border: 1px solid cadetblue;
}
```

Paragraph tag

span tag span tag

## display:inline

```
p{
    width: 100px;
    height: 100px;
    border: 1px solid cadetblue;
    display: inline;
}
span{
    width: 100px;
    height: 100px;
    border: 1px solid cadetblue;
}
```

Paragraph tag span tag span tag

## display:block;

```
p{
    width: 100px;
    height: 100px;
    border: 1px solid cadetblue;
}
span{
    width: 100px;
    height: 100px;
    border: 1px solid cadetblue;
    display: block;
}
```

Paragraph tag

span tag

span tag

## display:inline-block;

```
p{
    width: 100px;
    height: 100px;
    border: 1px solid cadetblue;
}
span{
    width: 100px;
    height: 100px;
    border: 1px solid cadetblue;
    display: inline-block;
}
```

Paragraph tag

span tag

## CSS Positions:

- Positions are used to place the html element at desired location.
- The position property defines how an element is placed in the document and how top/right/bottom/left values affect it.
- For positions we have position property it have some values
- Values are static, relative, absolute, fixed, sticky

### static (default)

- Elements are placed in the normal document flow.
- Top, right, bottom, left don't work.

### relative

- The element is placed **relative to its normal position**.
- You can shift it using top, left, right, bottom.

### absolute

- The element is placed relative to the nearest positioned ancestor (not the document).
- It will be based on its parent
- If no parent has position set, it will be relative to the <html>/body.

**fixed**

- The element is fixed to the viewport (browser window).
- It does not move even if the page is scrolled.

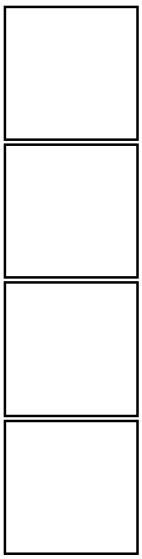
**sticky**

- Hybrid of relative + fixed.
- Acts like relative until you scroll past a point, then it "sticks" like fixed.
- When an element reaches its boundary point then the element becomes sticky

**Examples:**

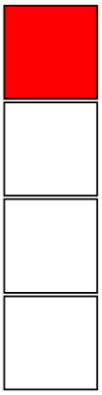
```html
<body>
    <div class="static"></div>
    <div class="relative"></div>
    <div class="absolute"></div>
    <div class="fixed"></div>
</body>
```

```css
div{
    height: 100px;
    width: 100px;
    border: 2px solid;
    margin: 2px;
}
```



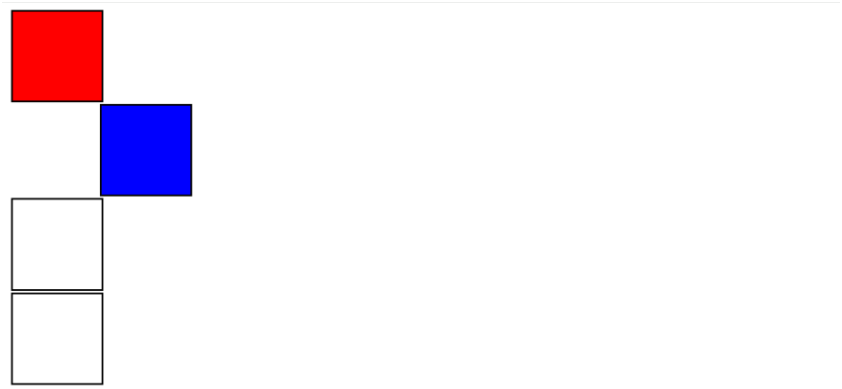
**Static:**

```
.static{
    background-color: red;
    position: static;
    top:50px;
}
```



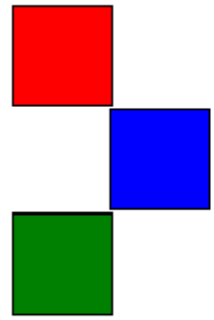
**relative:**

```
.relative{
    background-color: blue;
    position: relative;
    left: 100px;
}
```

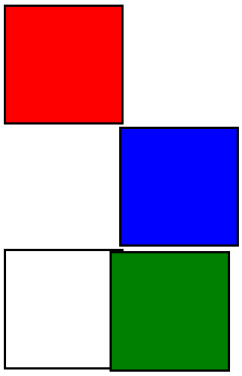


**absolute:**

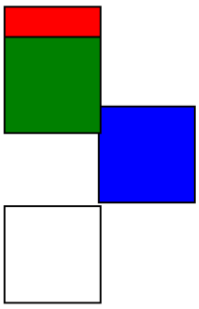
```
.absolute{
    background-color: green;
    position: absolute;
}
```



```
.absolute{
    background-color: green;
    position: absolute;
    left: 100px;
}
```



```
body{
    position: relative;
}
.absolute{
    background-color: green;
    position: absolute;
    top:30px;
}
```



## fixed:

```
<body>
    <p>Lorem</p>
    <p>Lorem</p>
    <p>Lorem</p>
    <p>Lorem</p>
    <p>Lorem</p>
    <p>Lorem</p>
    <p>Lorem</p>
    <div class="fixed"></div>
    <p>Lorem</p>
    <p>Lorem</p>
```

```
.fixed{
    background-color: orange;
    position: fixed;
}
```

## Sticky:

```
.sticky{
    background-color: chartreuse;
    position: sticky;
    top:30px
}
```

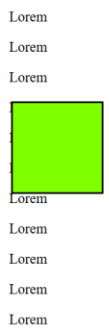
## z-index:

- The z-index property in CSS controls the stacking order of elements (which element appears on top of which).
- Elements with a higher z-index value are displayed in front of elements with a lower value.

**Basic rules for z-index:**

- z-index only works on positioned elements (relative, absolute, fixed, sticky). (If position: static, z-index does nothing.)
- Higher number = closer to the user.
- If two elements overlap, the one with greater z-index is visible on top.

```
p{
    z-index: 0;
    position: relative;
}
.fixed{
    background-color: chartreuse;
    position: fixed;
    z-index: 1;
}
```



```
p{
    z-index: 1000;
    position: relative;
}
.fixed{
    background-color: chartreuse;
    position: fixed;
    z-index: 1;
}
```

Lorem

Lorem

Lorem

Lorem

Lorem

Lorem

Lorem

Lorem

Lorem

Lorem